

entire `binarySearch()` function becomes $O(n \log n)$. If we can guarantee that `binarySearch()` will only ever be called with a sorted list for `haystack`, we can remove the `haystack.sort()` line and make `binarySearch()` an $O(\log n)$ function. This technically improves the function's efficiency but doesn't make the overall program more efficient, because it just moves the required sorting work to some other part of the program. Most binary search implementations leave out the sorting step and therefore the binary search algorithm is said to have $O(\log n)$ logarithmic complexity.

The Big O Order of Common Function Calls

Your code's big O analysis must consider the big O order of any functions it calls. If you wrote the function, you can just analyze your own code. But to find the big O order of Python's built-in functions and methods, you'll have to consult lists like the following.

This list contains the big O orders of some common Python operations for sequence types, such as strings, tuples, and lists:

`s[i]` reading and `s[i] = value` assignment $O(1)$ operations.

`s.append(value)` An $O(1)$ operation.

`s.insert(i, value)` An $O(n)$ operation. Inserting values into a sequence (especially at the front) requires shifting all the items at indexes above `i` up by one place in the sequence.

`s.remove(value)` An $O(n)$ operation. Removing values from a sequence (especially at the front) requires shifting all the items at indexes above `i` down by one place in the sequence.

`s.reverse()` An $O(n)$ operation, because every item in the sequence must be rearranged.

`s.sort()` An $O(n \log n)$ operation, because Python's sorting algorithm is $O(n \log n)$.

`value in s` An $O(n)$ operation, because every item must be checked.

`for value in s:` An $O(n)$ operation.

`len(s)` An $O(1)$ operation, because Python keeps track of how many items are in a sequence so it doesn't need to recount them when passed